CS 623 Final Project Report: Smart EV Charging Reservation Platform

Team Members: Sukhpinder Singh & Inderpal Singh

Submission Date: April 30, 2026

1. **Project Description**

The Smart EV Charging Reservation Platform is a hybrid cloud-native system designed to bridge the gap between physical charging infrastructure and user-accessible data. The project utilizes a Software-Defined Networking (SDN) approach to simulate a distributed network of charging stations. Each station acts as an "Edge" node, generating high-fidelity telemetry including battery State of Charge (SoC) and charging voltage.

2. **Functionality**

- **Real-Time Status Monitoring:** The system provides instantaneous feedback on station availability (Active/Inactive status).
- **Live Telemetry Streams:** Users and administrators monitor battery percentages and voltage levels with sub-2-second latency.
- **Time-Series Visualization:** A dynamic "Network-wide Voltage Trend" graph plots historical data points to analyze grid stability.

- **Edge-to-Cloud Integration:** Data is autonomously extracted from isolated Docker containers and synchronized to a central web service.

3. **System Workflow**

The system follows a structured data flow to simulate a cloud-based architecture:

- EV stations (Docker containers) generate real-time telemetry data
- A Bash pipeline extracts and merges data into a centralized JSON file
- The JSON file acts as the system's data storage layer
- json-server exposes the JSON data as RESTful API endpoints
- The React dashboard fetches and visualizes the real-time data

4. **Frontend, Backend, and Services**

- **Frontend:** Built with React.js. It utilizes functional components and React Hooks (useEffect, useState) for data polling and Recharts for the responsive time-series analytics.
- **Backend & API Bridge:** JSON-Server acts as the RESTful API Gateway, providing standardized HTTP endpoints for the frontend to consume edge data.
- **Simulation Infrastructure:** Containernet and Docker are used to virtualize the charging stations as SDN-enabled hosts, allowing for realistic network testing.
- **Data Orchestration:** A custom Bash-based pipeline manages the extraction and pruning of data from the Edge (Docker) to the Cloud (API).

5. **Performance Analysis**

- **Latency:** The end-to-end data loop (generation to visualization) maintains a consistent synchronization interval of ~2.0 seconds.
- **Scalability:** By utilizing a pruning pipeline, the system ensures that only the most recent 15 data points per station are processed by the API, preventing memory bloat and maintaining a lightweight footprint.
- **Data Integrity:** Unique ID stamping ensures that the time-series graph maintains chronological continuity without data collision.

6. **Project Evaluation** (What Worked & Issues)

- **What Worked Well:** The hybrid model proved highly effective. Using Containernet allowed for a realistic simulation of network latency that would be difficult to achieve with standard web tools. The React frontend successfully handled high-frequency state updates without UI flickering.
- **Issues Encountered:** * JSON Synchronization: We initially faced a "Position 17" syntax error caused by multiple stations writing to the same log. This was resolved by creating separate buffers (stn1.json, stn2.json) and merging them into a valid JSON object.
    - **Graph Visibility:** Initial attempts resulted in invisible lines due to a lack of historical context; we corrected this by implementing a 15-point "sliding window" history in the pipeline script.

7. **Cloud Concept Justification**

- Although implemented locally, this system simulates a cloud architecture by using distributed EV stations, a centralized API layer, and real-time data synchronization. This reflects key cloud computing principles such as scalability, modular services, and abstraction of infrastructure.

8. **Conclusion**

- This project successfully demonstrates a cloud-inspired EV charging monitoring system using a local simulation. The system achieves real-time data processing, modular architecture, and scalable design principles aligned with cloud computing concepts.

Installation and Execution Guide

1. **Prerequisites**

Ensure the following are installed on an Ubuntu Linux environment:

- **Containernet** (SDN Framework)
- **Docker** and **Node.js**
- **Global API Tool**: sudo npm install -g json-server

2. **Execution Steps**

Step 1: **Start the Topology**

sudo python3 ~/smart_ev_lab/smart_ev_sim.py

Step 2: Initialize Telemetry (Run inside containernet> prompt) Station 1:

station1 python3 -c "import time, json, random; f=open('/ev_sim_cloud_log.json','a');
[f.write(json.dumps({'station_id':'STN-001',
'battery_pct':round(random.uniform(60,95),1),
'voltage_v':round(random.uniform(230, 245), 2)})+'\n') or f.flush() or time.sleep(2) for _ in range(1000)]"
&

Station 2:

station2 python3 -c "import time, json, random; f=open('/ev_sim_cloud_log.json','a');
[f.write(json.dumps({'station_id':'STN-002',
'battery_pct':round(random.uniform(20,60),1),
'voltage_v':round(random.uniform(230, 245), 2)})+'\n') or f.flush() or time.sleep(2) for _ in range(1000)]"
&

Step 3: Launch API Bridge (New Terminal)

json-server --watch ~/smart_ev_lab/ev_sim_cloud_log.json --port 5000

Step 4: Start Sync Pipeline (New Terminal)

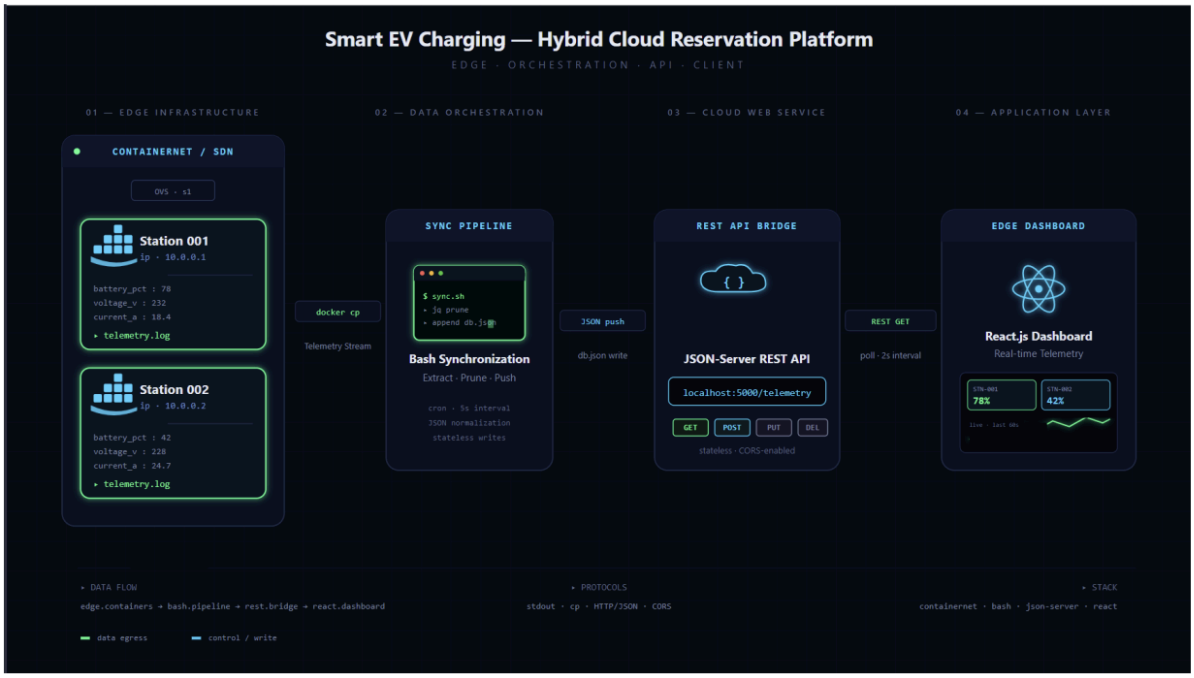
```
while true; do
sudo docker cp mn.station1:/ev_sim_cloud_log.json ~/smart_ev_lab/stn1.json
2>/dev/null
sudo docker cp mn.station2:/ev_sim_cloud_log.json ~/smart_ev_lab/stn2.json
2>/dev/null
echo '{"telemetry": [' > ~/smart_ev_lab/temp_log.json
{ tail -n 15 ~/smart_ev_lab/stn1.json 2>/dev/null; tail -n 15 ~/smart_ev_lab/stn2.json
2>/dev/null; } |
grep "{" | paste -sd, - >> ~/smart_ev_lab/temp_log.json
echo ']}' >> ~/smart_ev_lab/temp_log.json

mv ~/smart_ev_lab/temp_log.json ~/smart_ev_lab/ev_sim_cloud_log.json
sleep 2
done
```

Step 5: Launch Dashboard (New Terminal)

```
cd ~/smart_ev_lab/ev-dashboard && npm install && npm start
```

Design Rationale



Visual Encoding Rationale: > The architectural diagram utilizes specific color tracks to represent system logic. **Neon Green** paths denote data egress—representing the live telemetry flow from the edge nodes. **Cyan paths** represent the control and write paths, specifically the JSON integration into the API and the protocol semantics within the React dashboard.